



WhatsApp Business × OpenAI

AI inbox on the official Meta WhatsApp Cloud API: triage, search, reports and approved replies, with Arabic support and ChatGPT access over MCP.

What the client asked for

- Read and analyze incoming WhatsApp messages; flag important ones and those needing a reply.
- Search and summarize conversations; generate reports.
- Draft replies and send them only after approval, including from ChatGPT instructions.
- Official Meta Cloud API, production-ready and secure, with strong Arabic support.

Meta Cloud API

OpenAI

MCP / ChatGPT

Node.js + TypeScript

PostgreSQL + pgvector

React

Arabic / RTL

DOCUMENT

Initial approved build plan

STACK

Node/TypeScript, Postgres, React

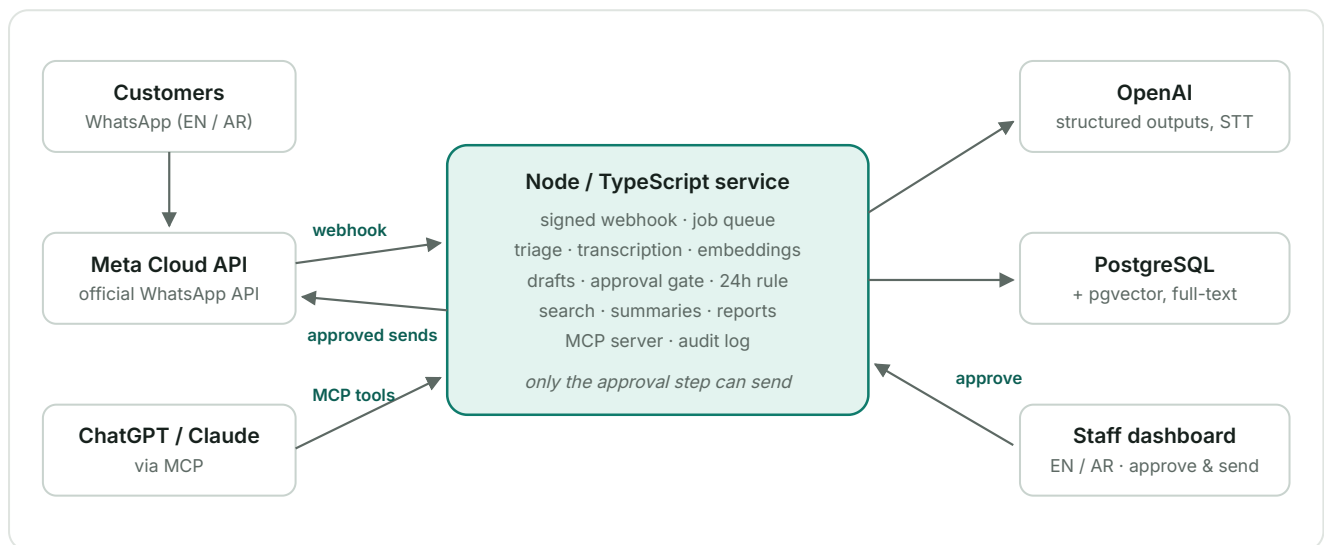
DATE

September 2026

1. Context

A business wants to connect its existing WhatsApp Business number to ChatGPT/OpenAI. The brief asks for more than a chatbot: an assistant that helps staff **work the inbox**. It should triage every message, surface what needs action, search and summarize history, produce reports, and draft replies that a person approves before sending. It must also let ChatGPT prepare or send messages on instruction, run on the **official Meta Cloud API**, and handle **Arabic** well.

This plan answers the client's questions (keeping the number, existing chats and groups, architecture, Arabic, costs, limitations) and lays out a production-ready build. It includes a demo mode that runs without Meta or OpenAI credentials, and a path to go live on Meta's test number.



2. Analysis of the brief

Can the existing WhatsApp number be kept?

Yes. There are two routes:

	Coexistence (recommended where available)	Full migration to the API
Phone app	Keeps working on the same number	Stops working for that number
Existing chats	Up to ~6 months of 1:1 history synced at onboarding	Not carried over (back up first)
Replies typed on the phone	Mirrored into the system (<code>smb_message_echoes</code>)	n/a
Trade-offs	Country availability varies; groups not synced; lower throughput; app must be opened every ~2 weeks	Full API throughput

Existing chats and WhatsApp groups

- **Existing 1:1 chats:** available through coexistence history sync. Anything older can be imported from WhatsApp's own "Export chat" `.txt` files, then indexed for search and summaries.
- **Existing groups:** the official API cannot read groups created in the app. Meta's Groups API only covers new groups the business creates through the API, with small participant caps. Stated upfront, because it is the most common false promise in these bids.
- **Unofficial libraries** (Baileys, whatsapp-web.js) are explicitly ruled out. They break Meta's terms and risk getting the number banned.

The 24-hour rule

Free-form replies are only allowed within 24 hours of the customer's last message. After that, only approved **templates** can be sent. The design enforces this, and the "send via ChatGPT" feature supports templates as well as free-form text.

Recommended architecture

Custom Node core Webhook, Postgres store, OpenAI calls and the approval workflow live in real code with a real database. Approvals, audit and security need this.	MCP server on top One integration lets ChatGPT, Claude and the OpenAI Responses API search, summarize, report and prepare messages.
n8n at the edges only Useful for side automations (e.g. emailing the daily report), not for the core logic.	Human approval gate No autonomous agent sends on its own. Every outbound message goes through approval.

Arabic support

- OpenAI models handle Modern Standard Arabic, dialects and Arabizi. Drafts reply in the customer's dialect.
- Search normalizes Arabic (diacritics, tatweel, alef/yaa/taa-marbuta variants, Arabic-Indic digits) and adds multilingual semantic search (`text-embedding-3-small`) alongside Postgres full-text search.
- Voice notes, very common in Arabic-speaking markets, are transcribed.
- The dashboard is fully right-to-left in Arabic.

Ongoing costs

Item	What to expect
Meta	Service replies inside the 24h window are free. Templates are charged per message by category (marketing, utility, authentication) and country.
OpenAI	Usage-based. Triageing a few thousand messages a day with a small model costs a few dollars a month; summaries, drafts and reports cost more.
Hosting	A small container host plus managed Postgres: roughly \$10–50 per month.
Support	Optional maintenance retainer.

Limitations, stated upfront

- No access to existing WhatsApp groups through the official API.
- The 24h window and Meta's template approval process.
- AI output always needs human review; the approval step exists for this.
- Coexistence restrictions, and Meta business verification can take several days.

Delivery milestones (~3–4 weeks)

#	Milestone	Scope
1	Cloud API onboarding & ingestion	Meta app and number, signed webhook, storage, history import
2	AI triage & search	Priority / needs-reply / intent, voice notes, Arabic-aware search, summaries
3	Drafts, approval & MCP	Approval dashboard, templates, 24h rule, ChatGPT connector
4	Reports & hardening	Daily/weekly reports, roles, audit, retention, deployment, handover docs

3. Build plan (Node.js / TypeScript + PostgreSQL)

Repository layout (pnpm workspace)

```
apps/server      Fastify API + webhook + worker + MCP endpoint
apps/web         React + Vite dashboard (RTL/LTR, English/Arabic)
packages/shared  zod schemas, types, Arabic normalization
migrations/      Postgres 16 + pgvector schema
scripts/         simulator (signed fake webhooks), demo seed, chat-export importer
docs/            proposal, architecture, security, deployment
Dockerfile, docker-compose.yml, .env.example, README with screenshots
```

Data model

Table	Purpose
contacts, conversations	Customer by WhatsApp ID; last_inbound_at drives the 24h window
messages	Unique wa_message_id (idempotency), direction, type, text, transcript, delivery status, full-text vector, embedding
message_analyses	Priority, needs-reply, intent, sentiment, language, action items, summary
drafts	Body or template + parameters; source (AI / staff / MCP); state pending → approved → sent rejected failed
templates, reports	Synced from the Graph API; generated reports with stats and narrative
audit_log, users	Every send, approval and MCP call; admin / agent / viewer roles

Background jobs run on **pg-boss**, so the queue also lives in Postgres and no Redis is needed.

Server modules

- **Webhook:** verification handshake; timing-safe HMAC check of X-Hub-Signature-256 over the raw body; fast 200 then queued processing. Handles messages, statuses, coexistence echoes and history.
- **Graph API client:** sends text, templates and media; downloads media; marks messages read; lists templates; retries on 429/5xx.
- **AI (OpenAI Responses API with structured outputs):** per-message triage, conversation summaries, reply drafts in the customer's language and dialect, voice-note transcription, embeddings.
- **Prompt-injection guard:** customer text is always passed as quoted data, never instructions, and the AI layer has no ability to send.
- **Hybrid search:** pgvector similarity fused with full-text ranking, with filters for contact, date and direction.
- **Reports:** volume, first-response time, unanswered items, top intents, action items, plus an AI-written narrative in English or Arabic.
- **Approvals:** the draft state machine is the only code path that sends, and it enforces the 24h window (templates only outside it).

- **MCP server:** tools `search_messages`, `list_conversations`, `get_conversation`, `summarize_conversation`, `generate_report`, `list_templates`, `draft_reply`, `request_send`. `request_send` creates a pending draft and returns an approval link.
- **Demo mode:** a mock WhatsApp client writes to an outbox table and a deterministic AI stub runs offline, so the whole product works without credentials.

Dashboard

- Inbox sorted by priority, with a "needs reply" queue.
- Conversation view with the AI summary and action items.
- Draft editor with Approve / Edit / Reject, and a template picker when the 24h window is closed.
- Search, reports, ChatGPT connection and audit pages; English/Arabic toggle with full RTL layout.

Build order

1. Workspace scaffold, Postgres + pgvector, schema and migrations.
2. Webhook, signature verification, normalization, persistence, simulator.
3. AI triage, embeddings, transcription, Arabic normalization.
4. Drafts, approval state machine, Graph API sending, template sync, 24h rule.
5. Search, summaries, reports.
6. MCP server.
7. Dashboard.
8. Documentation, screenshots and bilingual demo data.

Security

- HMAC webhook verification
- Secrets only in the environment
- Roles and a full audit log
- Rate limiting on API and MCP

- Configurable data retention
- No personal data in logs
- Idempotent processing on message ID
- Approval gate on every outbound message

4. Verification plan

- **Automated tests:**
 - webhook signature (valid, tampered, wrong secret);
 - payload normalization for text, audio, status, echo and history;
 - Arabic normalization;
 - draft state machine including the 24h-window rule;
 - hybrid search ranking;
 - MCP tools.
- **Static checks:** typecheck and lint.
- **End-to-end in demo mode:** seed data, send signed Arabic and English webhooks, confirm triage and drafts, approve one and see it in the outbox.
- **MCP:** point an MCP client at `/mcp`, call `search_messages` and `request_send`, and confirm the result is a pending draft, not a send.
- **UI:** browser screenshots of the dashboard in LTR and RTL.
- **Live:** Meta's test number and webhook; send a real WhatsApp message and approve a reply.

Outcome

The plan was built as specified and is running live.

- **Live deployment:** on Render, receiving real WhatsApp messages from Meta's test number through the official Cloud API.
- **56 automated tests** plus typecheck and lint (with a CI workflow), covering the full inbound pipeline, approvals and the 24h rule, Arabic search, reports and MCP over HTTP.
- **Bilingual dashboard:** English/Arabic with full RTL, dark mode and mobile layout.
- **MCP server:** ChatGPT or Claude can search, summarize, report and prepare replies; nothing is sent without human approval.
- **Deployment docs:** one-click Render Blueprint, environment template and troubleshooting guide for going live.